

Volume 16

Cyber Object Operating System (COOS)

Version: 1.0

Status: Draft

Vision

Everything inside Project Sentinel is an object.

Not just assets.

Everything.

Objects live inside one universal ecosystem.

Objects can:

- know themselves
- know their history
- know their relationships
- know their evidence
- know their permissions
- know their trust level
- know their lifecycle
- know their owner

This creates one unified architecture.

Chapter 1

Universal Object

Every object inherits from the same foundation.

SentinelObject

No exceptions.

Chapter 2

Everything is an Object

Examples

Infrastructure

- Domain
- Website
- Server
- Laptop
- Mobile
- Router
- Firewall

Identity

- User
- Group
- API Key
- Certificate

Security

- Finding
- Alert
- IOC
- Malware
- Threat

Operations

- Mission
- Report
- Workflow
- Case
- Investigation

Compliance

- Control
- Requirement
- Policy
- Exception

Knowledge

- Evidence
- Documentation
- Learning Article
- Best Practice

Chapter 3

Universal Object ID

Every object receives a globally unique identifier.

Examples

PS-DOM-000001

An object ID is never reused.

Chapter 4

Object Lifecycle

Every object follows a predictable lifecycle.

Created

Some objects may also enter:

Compromised

The lifecycle is visible to users.

Chapter 5

Object History

Objects never lose their history.

Example

Laptop

Purchased

Years later.

Everything is still available.

Chapter 6

Object Relationships

Objects automatically discover relationships.

Example

Employee

Impact analysis becomes much easier.

Chapter 7

Object Permissions

Objects have permissions.

Examples

Read

Update

Delete

Investigate

Approve

Export

Archive

Permissions are evaluated through the RBAC engine.

Chapter 8

Object Intelligence

Objects continuously accumulate knowledge.

Website

knows

- last scan
- technologies
- certificates
- vulnerabilities
- owner
- dependencies
- risk trend

Laptop

knows

- operating system
- installed software
- users
- security posture
- history
- trust profile

Every object becomes smarter over time.

Chapter 9

Object Communication

Objects never communicate directly.

Everything passes through the Event Engine.

Object

Loose coupling.

Easy maintenance.

High scalability.

Chapter 10

Object Versioning

Objects evolve.

Version example

v1

The platform always knows:

Who changed it?

When?

Why?

Approval?

Evidence?

Rollback?

Chapter 11

Object Ownership

Every object has ownership.

Examples

Technical Owner

Business Owner

Department

Country

Project

Environment

This supports enterprise governance.

Chapter 12

Object Health

Every object has a health profile.

Example

Laptop
Health

Not one score.
Several independent dimensions.

Chapter 13

Object Reputation

This is a new idea.
Objects develop a reputation over time.
Example
Server
Five Years

Another server
Frequent vulnerabilities

This helps prioritize work.

Chapter 14

Object Memory

Objects remember.
Not logs.
Knowledge.
Example
Website
CSP removed

The platform recognizes recurring patterns and suggests process improvements.

PS-ADR-0013

Everything is an Object

Status

Accepted

Decision

Project Sentinel shall implement one universal object architecture.

Reason

One architecture.

One API.

One lifecycle.

One permission model.

One event model.

One knowledge model.

This dramatically reduces complexity and makes the platform extensible.

NEW CORE ENGINE

Object Intelligence Engine (OIE)

Every object continuously updates itself from:

- Discovery Engine
- Endpoint Engine
- Mobile Engine
- Cloud Engine
- Web Engine
- Compliance Engine
- Threat Intelligence
- User Actions

Objects become living entities rather than static database records.

NEW CORE ENGINE

Object Relationship Engine (ORE)

The ORE continuously discovers relationships between objects.

Examples:

- Which APIs depend on a certificate?

- Which laptops belong to a department?
- Which websites share the same backend?
- Which cloud resources use the same identity?

Understanding these relationships improves risk analysis and impact assessment.

NEW CORE ENGINE

Cyber Knowledge Repository (CKR)

Earlier we discussed not relying on a traditional database as the heart of the platform.

I propose that the **Cyber Knowledge Repository** becomes the platform's primary storage model.

It stores:

- Objects
- Relationships
- Evidence
- Timelines
- Policies
- Trust Profiles
- Security DNA
- Reports
- Workflows

Under the hood, it can use reliable storage technologies (for example embedded databases, object stores, or graph-capable storage), but the rest of the platform interacts only with the Repository through defined interfaces. This keeps the architecture flexible and avoids coupling the core to a specific storage technology.

NEW IDEA

Cyber Security Language (CSL)

One idea that could become unique is creating a small domain-specific language for Project Sentinel.

Example:

MISSION ScanWebsite

Instead of clicking through many menus, users, automation, or CI/CD pipelines can describe security missions declaratively.

NEW IDEA

Sentinel Automation Studio

A visual workflow builder.

Users can create workflows like:

Website Added

No programming required for common automations.

A long-term engineering vision

I believe that after we finish approximately **Volume 30**, we will have a complete, internally consistent specification. The remaining volumes will focus on implementation details, APIs, testing, deployment, operations, and documentation.

At that stage, we can begin building Project Sentinel module by module in Python with confidence because every major architectural decision will already be documented.

One proposal before we continue

I would like us to adopt a project motto that appears on the first page of every Blueprint volume:

Project Sentinel

Know Everything. Trust Evidence. Secure with Confidence.

That motto reflects everything we have built so far:

- **Know Everything** → Discovery, Asset Intelligence, Knowledge Graph.
- **Trust Evidence** → Explainable, auditable, evidence-based conclusions.
- **Secure with Confidence** → Guided remediation, verification, and measurable improvement.

It is ambitious, but every part of it is technically achievable with the architecture we are designing.

====[USER]=====

Good and continue

====[ASSISTANT]=====

Excellent. I think we are now designing something that is much larger than a cybersecurity application.

We are designing an **engineering platform** that could realistically evolve over 10-20 years.

From now on, I want every new capability to satisfy three requirements:

1. **Technically feasible** (we can build it with Python, Linux, open standards, and documented APIs).
2. **Useful** (it solves a real security problem).
3. **Elegant** (it remains simple for the user).

That will prevent Project Sentinel from becoming unnecessarily complex.

PROJECT SENTINEL BLUEPRINT

Volume 17